

Processos e Threads

1. Identificação

- **Nome dos Integrantes:** Thiago Freitas, Caio Teixeira, Isabella Fernandes, Gessica Nogueira, Lucas Basilio, Guilherme Diniz
- **Disciplina:** Computação Paralela e Distribuída
- **Linguagem e Versão do Python:** Python 3.14.7 (CPython 64-bit)

Configuração do Computador Utilizado:

Característica	Especificação Real
Processador (CPU)	AMD Ryzen 7 5700X 8-Core Processor
Núcleos Físicos	8 núcleos
Processadores Lógicos	16 threads (SMT ativado)
Memória RAM	31.92 GB DDR4
Sistema Operacional	Windows-11-10.0.26200-SP0

2. Código-Fonte

2.1 Programa Sequencial (sequencial.py)

```
import time

def eh_primo(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

if __name__ == "__main__":
    inicio = time.perf_counter()
    numeros = range(2, 200000)
    quantidade = 0
    for numero in numeros:
        if eh_primo(numero):
            quantidade += 1
    fim = time.perf_counter()
    print("Quantidade de primos:", quantidade)
    print("Tempo:", fim - inicio, "segundos")
```

2.2 Programa com Processos (processos.py)

```
import time
from multiprocessing import Process, Queue

def eh_primo(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

def processar(inicio, fim, fila):
    quantidade = 0
    for numero in range(inicio, fim):
        if eh_primo(numero):
            quantidade += 1
    fila.put(quantidade)

if __name__ == "__main__":
    inicio = time.perf_counter()
    fila = Queue()

    p1 = Process(target=processar, args=(2, 50000, fila))
    p2 = Process(target=processar, args=(50000, 100000, fila))
    p3 = Process(target=processar, args=(100000, 150000, fila))
    p4 = Process(target=processar, args=(150000, 200000, fila))

    for p in (p1, p2, p3, p4): p.start()
    for p in (p1, p2, p3, p4): p.join()

    quantidade = sum(fila.get() for _ in range(4))
    fim = time.perf_counter()
    print("Quantidade de primos:", quantidade)
    print("Tempo:", fim - inicio, "segundos")
```

2.3 Programa com Threads (threads.py)

```
import time
import threading

def eh_primo(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

resultados = []

def processar(inicio, fim):
    quantidade = 0
```

```

for numero in range(inicio, fim):
    if eh_primo(numero):
        quantidade += 1
    resultados.append(quantidade)

if __name__ == "__main__":
    inicio = time.perf_counter()

    threads = [
        threading.Thread(target=processar, args=(2, 50000)),
        threading.Thread(target=processar, args=(50000, 100000)),
        threading.Thread(target=processar, args=(100000, 150000)),
        threading.Thread(target=processar, args=(150000, 200000))
    ]

    for t in threads: t.start()
    for t in threads: t.join()

    quantidade = sum(resultados)
    fim = time.perf_counter()
    print("Quantidade de primos:", quantidade)
    print("Tempo:", fim - inicio, "segundos")

```

3. RESULTADOS EXPERIMENTAIS

Parâmetros da execução: intervalo de 2 até 200.000 (199.998 números analisados). Total de primos encontrados em todas as execuções: 17.984 primos (corretude validada). Tempo sequencial de referência: 0.1945 s. Fórmula: $\text{Speedup (S)} = \text{Tempo_Sequencial} / \text{Tempo_Paralelo}$.

Estratégia	Fluxos	Tempo (s)	Speedup	Observações sobre Utilização da CPU
Sequencial	1	0.1945 s	1,00x	100% de 1 núcleo lógico (~6,25% da CPU global). Sem concorrência.
Processos	1	0.3109 s	0.63x	1 processo ativo. Custo de inicialização de processo (spawn) no Windows.
Processos	2	0.2365 s	0.82x	2 núcleos lógicos a 100% (~12,5% da CPU). Divisão de carga computacional.

Processos	4	0.1940 s	1.00x	4 núcleos lógicos a 100% (~25% da CPU). Ganho computacional compensa o overhead.
Processos	8	0.2064 s	0.94x	8 núcleos físicos operando a 100% (~50% da CPU). Estabilização por custo de IPC e spawn.
Threads	1	0.1988 s	0.98x	1 thread de cálculo. Desempenho equivalente ao sequencial.
Threads	2	0.2004 s	0.97x	2 threads sob 1 único núcleo ativo. O GIL impede paralelismo real de CPU.
Threads	4	0.2047 s	0.95x	4 threads disputando a mesma trava (GIL). Sobrecarga de trocas de contexto.
Threads	8	0.2047 s	0.95x	8 threads disputando 1 interpretador. Utilização de apenas 1 núcleo efetivo por vez.

4. Análise

Qual a diferença entre processo e thread?

Um processo é uma unidade independente de execução gerenciada pelo sistema operacional dotada de espaço de endereçamento de memória virtual próprio, descritores de arquivos e ambiente isolado. Essa separação garante proteção e estabilidade, mas acarreta maior sobrecarga de memória e exige comunicação via IPC (pipes, sockets ou filas). Uma thread é uma linha de fluxo interna ao processo ('processo leve'). Threads do mesmo processo compartilham a mesma memória (código, variáveis globais e heap), possuindo individualmente apenas registradores e pilha (stack). São muito mais rápidas de criar e alternar contexto, mas demandam mecanismos rigorosos de sincronização para prevenir condições de corrida.

Como o trabalho foi particionado?

O trabalho utilizou uma decomposição de domínio de dados em blocos contíguos (particionamento estático). O intervalo total de 199.998 números [2, 200.000) foi dividido em fatias homogêneas de tamanho igual de acordo com a quantidade de fluxos. Cada fluxo recebeu os limites (início, fim) da sua fatia, executou localmente o teste de primalidade e depositou sua contagem parcial em uma estrutura coletora (fila Queue para processos ou lista compartilhada resultados para threads), permitindo ao fluxo principal somar os totais.

Por que o problema é adequado à execução paralela?

Porque é um problema com característica embarçosamente paralela (embarrassingly parallel): a avaliação de primalidade de qualquer número N depende exclusivamente dos seus próprios divisores até $\text{floor}(\sqrt{N})$. Não há dependências de dados entre números distintos, não ocorre concorrência de variáveis compartilhadas durante o cálculo e a comunicação se dá uma única vez ao término da computação.

Qual foi o comportamento da versão com processos?

Apresentou capacidade de paralelismo real de hardware. Como cada processo conta com seu próprio interpretador Python independente e seu próprio GIL, o sistema operacional pôde escaloná-los para núcleos físicos distintos do AMD Ryzen 7 5700X. No entanto, para 1 processo, o tempo foi superior ao sequencial devido ao custo de inicialização de processos no Windows via spawn. Com 2 e 4 processos o tempo caiu, e com 8 processos estabilizou em 0,2064 s devido ao overhead de inicialização e IPC em uma carga tão rápida.

Qual foi o comportamento da versão com threads?

Apresentou estagnação de desempenho (tempos próximos de 0,20 s, sem ganho em relação ao sequencial). O motivo foi a contenção pelo Global Interpreter Lock (GIL) do CPython, que serializa a execução de bytecode entre threads em cargas CPU-bound. As threads concorreram em fatiamento de tempo sob um único núcleo efetivo, gerando sobrecarga adicional de chaveamento de contexto sem paralelismo real de hardware.

O aumento de processos ou threads sempre produziu ganho?

Não. Nas threads, aumentar fluxos apenas aumentou o tempo total devido à concorrência pela trava do GIL. Nos processos, houve redução de tempo até 4 fluxos, mas a passagem de 4 para 8 fluxos manteve o tempo estagnado, pois a sobrecarga de inicializar 8 processos pelo Windows equilibrou os ganhos da partição dos dados.

Quais fatores limitaram o desempenho?

1. O GIL do Python nas threads, que impede a execução paralela de bytecode;
2. A sobrecarga de inicialização de processos no Windows (spawn overhead), que requer criar novas instâncias do interpretador e carregar bibliotecas;
3. A curta duração da tarefa: os 200.000 números foram executados em menos de 0,2 s pelo Ryzen 7 5700X, amplificando o impacto percentual do custo fixo de criação de fluxos;
4. O desbalanceamento algorítmico natural: testar números maiores (perto de 200.000) exige mais divisões aritméticas que números menores (perto de 2);
5. A serialização de dados (pickle) na fila de comunicação IPC.

Qual estratégia apresentou comportamento diferente da outra?

A versão com processos comportou-se de maneira diametralmente oposta à versão com threads: enquanto os processos conseguiram romper o bloqueio do interpretador e ocupar núcleos físicos de hardware distintos reduzindo o tempo de cálculo, as threads ficaram confinadas a um único núcleo por vez em virtude do GIL, gerando execução concorrente serializada.

Qual relação existe entre os resultados e a arquitetura do computador utilizado?

A CPU AMD Ryzen 7 5700X conta com 8 núcleos físicos e 16 processadores lógicos de altíssimo IPC na microarquitetura Zen 3. Devido à sua velocidade, a carga total durou uma fração mínima de segundo (~0,19 s). Isso fez com que o tempo fixo de alocação de processos no Windows pesasse proporcionalmente sobre o tempo total. Os 8 núcleos físicos permitiram que os 8 processos fossem alocados diretamente em núcleos dedicados sem disputa SMT; para que esses núcleos exibam aceleração elevada (como 6x ou 7x), a carga precisa ser ampliada (ex.: 2.000.000 de números), onde o tempo de cálculo em segundos supere os milissegundos de overhead.

O que deveria ser investigado antes de transformar uma aplicação sequencial em uma aplicação paralela?

Deve-se investigar:

- O perfil da carga: se é intensiva em processamento (CPU-bound) ou em entrada/saída (I/O-bound);
- A independência das tarefas e presença de dependências de dados;
- As características da linguagem e plataforma (como o impacto do GIL no CPython);
- A fração paralelizável do algoritmo e a relação custo/benefício (Lei de Amdahl), verificando se o tempo economizado no cálculo é maior que o overhead de criação e sincronização dos fluxos;
- A quantidade de núcleos físicos disponíveis na máquina para evitar sobre-subscrição (oversubscription).